

第十八单元：正则表达式

1. 简介

1.1. 什么是正则表达式

- 正则表达式，也叫规则表达式，是对字符串操作的一种逻辑公式。
- 在 JavaScript 中，正则表达式是对象，可以用于字符串的匹配、搜索、替换和分割操作

1.2. 正则表达式的使用场景

- 验证输入格式（如邮箱、电话号码、身份证号等）
- 从字符串中提取特定信息（如从 URL 中提取参数）
- 替换字符串中的特定内容（如敏感词过滤）
- 基于特定模式分割字符串

1.3. 正则表达式创建：

1. 字面量：`var reg = /规则/修饰符`
2. 实例化：`var reg = new RegExp(规则 , 修饰符 )`

2. 基本语法

2.1. 修饰符

<code>i</code>	忽略大小写
----------------	-------

g	全局匹配
m	多行匹配

2. 2. 元字符

\d	匹配数字
\D	匹配非数字
\w	匹配数字字母下划线
\W	匹配非数字字母下划线
\s	匹配空字符
\S	匹配非空字符
\b	匹配单词边界
\B	匹配非单词边界
[\u4e00-\u9fa5]	匹配中文

2. 3. 特殊字符

\	转义字符, 将在正则中有特殊含义的字符转为普通字符
^	以什么开始

\$	以什么结束
[^]	取反, 除了中括号里以外的字符
.	除了换行以外的任意字符
	或
()	分组
[]	中括号中的字符匹配任意一个

2.4. 字符集合与范围

使用方括号[]定义字符集合，可以匹配集合中的任意一个字符：

- [abc]：匹配 "a"、"b" 或 "c"
- [a-z]：匹配任意小写字母
- [A-Z]：匹配任意大写字母
- [0-9]：匹配任意数字
- [^abc]：匹配除"a"、"b"、"c"外的任意字符（^在方括号内表示取反）

2.5. 限定符

{n, m}	匹配 n-m 个
{n}	匹配 n 个
{n, }	最少匹配 n 个

+	一个或多个，相当于 {1,}
*	0 个或多个，相当于 {0,}
?	0 个或一个，相当于 {0, 1}

3. 使用正则

3.1. 正则表达式方法

- `test()`:
 - **功能：**测试字符串是否符合指定的正则表达式模式，返回布尔值。
 - **示例：**

```
var regex = /hello/;  
var result = regex.test('hello world');  
// result 为 true, 因为字符串包含"hello"
```

- `exec()`:
 - **功能：**在字符串中执行匹配搜索，并返回匹配结果；结果是一个数组。
 - **注意：**该方法调用一次执行一次，如果没有加全局匹配，每次都找到第一项返回，
 - 如果 加了全局匹配，从上一次匹配 之后开始查找，找不到返回 `null`。
 - **示例：**

```
var regex = /hello/;
var result = regex.exec('hello world');
// result 为 ["hello"], 包含匹配的文本和相关信息

// 需求 再长字符串中匹配 1*0 示例: 150 180 160...
var str = "150de1806cd17cd0njk130";
var numReg = /1\d0/;
var res1 = numReg.exec(str);
var res2 = numReg.exec(str);
console.log(res1 , res2 )的结果都是[150] [150]

var str = "150de1806cd17cd0njk130";
var numReg = /1\d0/g;
var res3 = numReg.exec(str);
var res4 = numReg.exec(str);
console.log(res3 , res4)结果是[150] [180]
```

3.2. 字符串方法

- **match():** 返回字符串中的匹配结果。
 - **功能:** 查找并返回所有匹配。没有加全局匹配, 默认返回匹配 第一项, 加了全局匹配返回所有匹配项组成的数组
 - **示例:**

```
var result = 'hello world'.match(/hello/);
// result 为 ["hello"]
```

```
// 需求 再长字符串中匹配 1*0 示例: 150 180 160...
var str = "150de1806cd17cd0njk130";
var numReg = /1\d0/;
var res1 = str.match(numReg);
var res2 = str.match(numReg);
console.log(res1 , res2 ) // [150] [150]
var numReg = /1\d0/g;
var res3 = str.match(numReg);
console.log(res3) // [150,180,130]
```

- **search()**

- **功能:** 返回模式在字符串中的位置。
- **示例:**

```
var result = 'hello world'.search(/world/);
// result 为 6, 因为"world"从索引 6 开始
```

- **replace()**

- **功能:** 替换字符串中的匹配内容。
- **注意:** 如果 replace 的第二个参数是一个函数
 - 1 匿名函数执行多少次, 取决于正则能在字符串中捕获多少次
 - 2 每一次执行函数, arguments 值和通过 exec 捕获到的内容很相似
 - 3 return 返回值就是要去替换的内容

- 示例:

```
var str = "我爱中国，我是中国人";
console.log(str.replace("中国","china"))    // "我爱 china，我是中国人"
console.log(str.replace(/中国/g,"china"))    // "我爱 china，我是 china 人"

// replace() 方法 第二个参数可以接收一个函数，该函数接收两个参数。
//          参数一表示匹配正则的项，参数二表示匹配项的下标
// 把 get-element-by-id 转成 getElementById
// 之前 借助 - 转成数组 在循环遍历数组 下标 0 转大写 拼接返回
var str = "get-element-by-id";
var res = str.replace(/-[a-z]/g,function($0,$1){
    // console.log($0 , $1)  0 是匹配正则的项    1 是匹配的下标
    return $0[1].toUpperCase()
})
console.log(res)    // getElementById
```

- split()

- 功能：使用模式分割字符串。

- 示例:

```
var result = 'hello,world'.split(/,/);
// result 为 ["hello", "world"]
```

4. 常见正则

4.1. 手机号正则

- 以数字 1 开头

- 第二位不能是[012]
- 后 9 位任意数字

```
var reg=/^1[3-9]\d{9}$/;
```

4.2. 邮箱正则

- 开头可以多个是数字字母下划线
- 中间有@符
- @符后也是多个数字字母下划线
- 接着匹配.
- 后面跟常见尾缀(**com** | **cn** | **net**)

```
var reg =/^\\w+@\\w+\\. (com|cn|net)$/;
```

4.3. 网址正则

- http 开头, s 可有可无, 后面是://
- @前是数字字母下划线
- @后. 前是数字字母下划线
- . 后是尾缀

```
var reg=/^https?:\\/\\/w{3}\\w+\\. (com|cn|net)$/;
```


4. 4. 匹配前后空格

- 只匹配开头或结尾，不管中间

```
var reg = /^\s*|\s*$/g;
```

5. 分组

1、群组分块

```
/ab{3}/    //b 重复 3 次  
/(ab){3}/  //ab 整体重复 3 次
```

2、群组获取

```
var str="23+36+28+21";  
// 全局匹配 分组匹配一个数字或多个数字前面有+ 或 没有+的字符  
console.log(str.match(/\+(\d+)/g))  // ['23', '+36', '+28', '+21']  
console.log(str.match(/^(\d+)\+(\d+)$/)) // null  
//匹配以一个或多个数字开头的字符为一组，然后匹配+字符，再匹配一个或多个数字结束；  
//而该字符串中出现了多个+数字所以返回 null  
//正确写法应该是: /^(d+)(\+d+)$/g  
console.log("abac".match(/a(b)a(c)$/))  
// ['abac', 'b', 'c', index: 0, input: 'abac', groups: undefined]
```

3、断言

```
m(?=n)  m 后面必须紧跟一个 n，查找到的是 m 后置肯定断言  
var str="abacad";  
console.log(str.replace(/a(?=c)/g,"0"))  // "ab0cad"  
//全局匹配 断言 a 的后面紧跟有 c 字符时，将该字符 a 替换成 0  
  
m(?!n)  m 后面不能紧跟一个 n，查找到的是 m 后置否定断言
```

```

var str="abacad";
console.log(str.replace(/a(?!c)/g,"0"))    // "0bac0d"
// 全局匹配 断言 a 后面紧跟不是 c 字符时, 将该字符 a 替换成 0

(?:<=n)m  m 前面必须紧跟一个 n, 查找到的是 m 前置肯定断言
var str="cabada";
console.log(str.replace(/(?:<=b)a/g,"0"))    // "cab0da"
// 全局匹配 断言 a 的前面紧跟有 b 字符时, 将该字符 a 替换成 0

(?:<!n)m  m 前面不能紧跟一个 n, 查找到的是 m 前置否定断言
var str="cabada";
console.log(str.replace(/(?:<!b)a/g,"0"))    // "c0bad0"
// 全局匹配 断言 a 的前面紧跟不是 b 字符时, 将该字符 a 替换成 0

```

4、群组重复

(内容)\1 内容重复 1 次 (内容)\1+ 内容重复 1 次以上 (内容)\1* 内容重复 0 次以上

```

var str="aaaaaaabbbbbcccccccccccd";
console.log(str.match(/(\w)\1+/g))
console.log(str.match(/(\w)\1*/g))

```

5、群组变量

(?<变量名>内容) 正则内容赋值给变量名, match matchAll, exec 中的 groups 都可以获取

```

var str="34+27";
console.log(str.match(/(?<a>\d+)\+(?<b>\d+)/))
console.log(str.match(/(?<a>\d+)\+(?<b>\d+)/).groups.b)

```

6、群组变量调用

\k<变量名> \k<变量名>可以在正则中调用群组变量

```
var str="abc!abc";  
console.log(/(?<a>\w+)!\k<a>/.test(str))
```

6. 总结

1. 修饰符: **i** 忽略大小写 **g** 全局匹配
2. 元字符: **\d** 数字 **\w** 数字字母下划线 **\s** 空字符 **\b** 单词边界
3. 特殊字符: ****转义 |或 []任意一个 [^]除了 ^开头 \$结尾 .任意 ()分组
4. 限定符: **+** 1 或多 ***** 0 或多 **?** 0 或 1 **{}**
5. 正则方法: **test()** 返回布尔值 **exec()** 返回数组, 不支持全局匹配
6. 字符串方法: **replace()** 替换 **match()** 返回数组, 支持全局匹配

7. 案例（[链接](#)）

网易通行证

通行证用户名:	admin @163.com	请输入4-12位用户名（首位为字母）
登录密码:		密码为6-16位
重复登录密码:		请再次输入密码
性别:	☒ 男 ☐ 女	
真实姓名:		请输入您的真实姓名
昵称:		请输入昵称
关联手机号:		请输入关联手机号
保密邮箱:		请输入保密邮箱
注册		

"3[2[ab]2[2[e]3[df]]]--

>"ababeedfdfdfeedfdfdfeedfdfdfababeedfdfdfeedfdfdfeedfdfdfababeedfdfdfeedfdfdfeedfdfd"